

Semi-constrained Path Planning for Industrial Robot with External Axis

Yu Sun

Department of Electrical Engineering
Shanghai Jiao Tong University
Shanghai, China
tuzkisun@sjtu.edu.cn

Xiangqun Meng

Shanghai Fangling Computer Software
Co.,Ltd
Shanghai, China
meixiqu@163.com

Miaocheng Yu

Shanghai Fangling Computer Software
Co.,Ltd
Shanghai, China
64774953@qq.com

Houjun Tang

Department of Electrical Engineering
Shanghai Jiao Tong University
Shanghai, China
hjtang@sjtu.edu.cn

Abstract—To solve the problem that the working space of the industrial robot is limited by the arm length, and the flexibility is limited by the number of joints, the robot system with external axes is necessary. This paper proposes a novel sampling-based path planning method to control the industrial robot and external axes simultaneously. An external axis space is proposed for sampling external axis value to solve task-constrained path planning problems in practical industrial applications. The configuration space of a robot system with external axes is decoupled to extract task space independent of external axis space, based on the semi-constrained characteristic of typical Cartesian curve tracking tasks. The task space is defined by the position and orientation parameter of the robot's end-effector. As a result, a sampling space, which meets the task-constraint, can be found. Analytical Inverse Kinematics (IK) algorithm is used to map the samples back to the task-constrained configuration space without iterative modifications. Simulation studies on practical path planning tasks under constraints prove the effectiveness of the proposed method.

Keywords—industrial robot, external axis, task-constrained path planning, sampling algorithm

I. INTRODUCTION

Robots have increasingly been employed to accomplish complicated tasks in industrial domains such as arc welding, painting, and cutting, as it offers substantial cost benefits compared to the traditional five-axis CNC machine, and they need less floor space, whilst allowing production layouts to be optimized – thus saving both production space and cost. Industry and academia have been committed to improving the efficiency of industrial robots, especially in the complicated application fields. Sophisticated path planning methods of the industrial robot system are required to cope with practical industrial tasks, especially in the application field with task constraints, such as orientation constraint, path constraints, and obstacle avoidance constraints, also known as Task-Constrained Path Planning (TCPP).

Initially, academics mainly focus on Point to Point (PTP) path planning problem, with only constraints of the initial point and the goal point. The main concern is to find a valid path between two points, the method to solve such problem includes A* algorithm, ant colony algorithm (AG) [1], genetic algorithm (GA) [2], artificial potential field method (APF) [3], Gradient Projection Method (GPM), rapidly-exploring random trees (RRT) [4-7] method, etc. Nevertheless, when it comes to tasks with multiple constraints, some of these methods are proved to be inappropriate for realistic path planning.

Firstly, the above methods will be troubled by the singularity problem if the singular points are not pre-determined and concerned. Secondly, some methods such as AG, GA, APF, GPM would suffer from the local minima problem. When multiple constraints must be considered, the above problems are even worse. Also, some approaches would suffer from long computation time(A*), slow convergence speed (AG and GA) problem when degrees of freedom (DOF) of the robot are high.

However, some sample-based methods, such as RRT, have certain advantages in the application field with multiple constraints. Due to the inherent adaptability of such methods, task constraints can be easily incorporated into the sampling phase of the algorithm. Sampling-based algorithms usually sample valid robot configurations randomly and form a graph of proper motions [8]. These algorithms provide resolution completeness: A path will be found if the discretization is fine enough [9]. Sampling-based motion planning algorithms can effectively solve the motion planning problems in various environments with minimal changes, including ultra-high-dimensional systems.

Some studies use sampling-based methods to solve the TCPP problem of a single manipulator with multiple axes. Some ideas use Jacobian pseudoinverse projection techniques to map the sampling configurations in the configuration space back into its task constrained subspace [10-13]. Such methods need iterative modifications to eliminate the task space error, which increases the calculation time. Oriolo et al. introduced direct sampling in the task-constrained configuration space. A 4-DOF planar arm [14] was used to implement the TCMP by sampling in the task-constrained configuration space at the position level.

However, limited by the mechanical structure of the robot, it cannot handle some difficult industrial tasks independently, such as cutting large materials beyond the reach of the robot or cutting parts with complex surfaces. In such scenarios, robots need to work with the help of the external axes (e.g., gantry or rotation axis), which makes it difficult to develop IK algorithms for robotic systems with complex kinematics. Usually, robots and external axis will work independently, causing interruptions during the working process and a decline in accuracy and production quality.

This paper proposes a sample-based semi-constrained path planning method designed for the industrial robot system with external axes. The proposed method can be used in Cartesian curve tracking tasks, the most common constraints in industrial applications (e.g., plasma cutting and painting tasks

in manufacturing), which needs the robot's end-effector to follow a pre-planned Cartesian path. Also, this method can directly sample configurations in the task-constrained configuration space at the position level, without iterative modifications and singularity issues. Moreover, it can control the industrial robot system with external axes without interruption problems during the working process. Firstly, the sampling processes are progressively performed to reduce the difficulty of generating IK solutions. The sampling process for the manipulator will be started after the sampling process is done for the external axis. Secondly, we will leverage the redundant DOF of robots to directly generate configurations that satisfy the task constraints. After the sampling and IK generation processes, the shortest path algorithm is used to generate the optimum path in the joint space.

II. PROBLEM DEFINITION

In a d -dimensional configuration space (also called joint space), $\mathbf{C} \in \mathbf{R}^d$, where d equals the total DOF of the robot system, including the external axis. $c \in \mathbf{C}$ represents a robot system's configuration, such as joint angles and external axis angles/positions. To consider the surrounding obstacles and other task constraints in the realistic applications, constraints function $F(c: \mathbf{C} \rightarrow \mathbf{R}^k)$ is usually defined in Cartesian space, where k means the number of constraints. A configuration c satisfies the given constraint function when $F(c) = 0$, which means the constraint function defines a $(d-k)$ -dimensional manifolds in $\mathbf{C}$ -space, i.e., $\mathbf{M} = \{c \in \mathbf{C} | F(c) = 0\}$. In the general Cartesian curve tracking task, the goal of path planning is to search a continuous path $\sigma \subset \mathbf{M}$, which meets all task constraints.

III. SEMI-CONSTRAINED PATH PLANNING ALGORITHM

In this section, we will present our novel path planning method called the semi-constrained path planning algorithm. To solve the path planning problem for the robot system with the external axis under the Cartesian curve tracking task, we first generate configurations for each workspace path point directly on the constraint manifolds. Then we build the $\mathbf{C}$ -space graph, which connects all of the valid configurations along the workspace path. Based on the $\mathbf{C}$ -space graph, we can search for the optimal path. The proposed algorithm comprises the following components:

A. Semi-constrained Task Space and External Axis Space

In the case of executing curve tracking tasks in the workspace, the task constraint is usually pre-determined by CAD tools or parametric equations. Firstly, we need to discretize the continuous workspace path. By sampling a parameterized continuous workspace path at equal intervals, we can get a set of path points P . In typical industrial application scenarios such as cutting, welding, and spraying, the execution curve does not have to pass precisely through these points, but there is an error that may be assigned by specifying a prescribed tolerance. For example, when executing spraying tasks, the robot's end-effector can rotate around the Z_t axis freely without affect task execution. Due to the existence of tolerance in the workspace path, we can call it the semi-constraint path, as shown in Fig. 1.

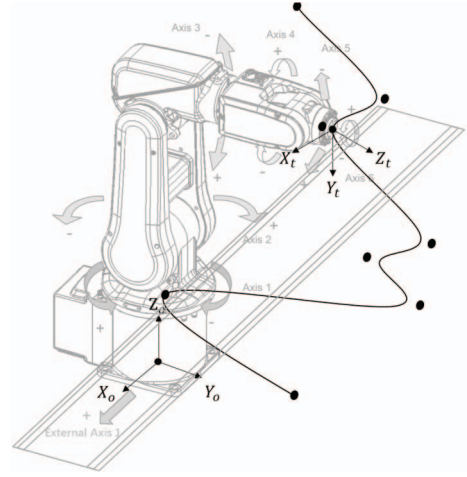


Fig. 1. Illustration of the semi-constraint path

Sample within the given tolerance for each semi-constraint path point $p_t (p_t \in P)$, we can get a set of Cartesian points which satisfy the task constraints. Therefore, the semi-constraints task space T is defined by the variables, x, y, z, R_x, R_y, R_z representing the position and orientation of the end effector.

$$T = \{(x, y, z, R_x, R_y, R_z | x \in [x - \frac{x_{tol}}{2}, x + \frac{x_{tol}}{2}], \dots)\} \quad (1)$$

Due to the existence of redundant DOF in the robot system with external axes, the configuration of the system cannot be determined with given position and orientation of the end effector. In order to leverage the structure feature of the industrial robot to reduce the difficulty of searching IK solutions for the entire robot system with the external axis, the system model is divided into two parts, the robot, and the external axis. Each part is sampled independently. Based on this feature, the whole configuration space can be decoupled to extract an external axis space independent of the task space, as shown in Fig. 2.

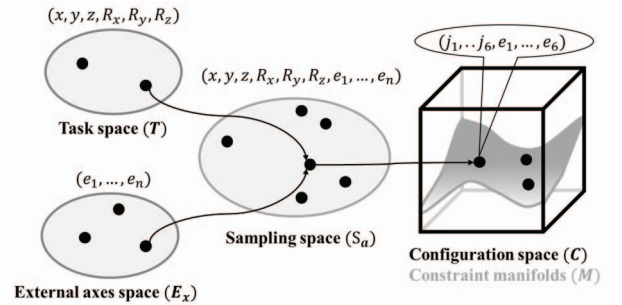


Fig. 2. Configuration generation process under task constraints

Therefore, the external axis space is defined by the variables, $e_1, \dots, e_n$, which control the position and orientation of the external axis.

$$E_x = \{e_1, \dots, e_n | e_1 \in [e_{1min}, e_{1max}], \dots\} \quad (2)$$

The dimension of the external axis space is equal to the DOF of the external axis.

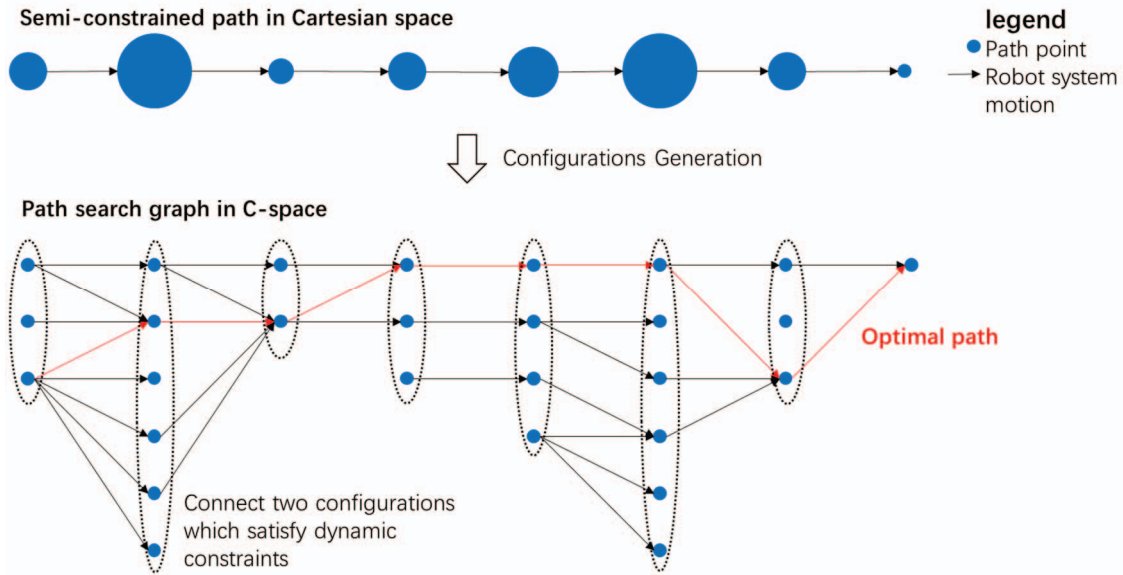


Fig. 3. Schematic diagram of semi-constrained path planning algorithm

B. Sampling Space

Because the external axis space is decoupled from the task space, A sampling point in the external axis space and an assigned position and orientation of the end effector in the task space can be used to calculate the robot configuration. Therefore, a Sampling Space, S_a , can be defined as:

$$S_a = T \times E_x \quad (3)$$

The subroutines of the sampling process include:

- 1) For each Cartesian path point, sample the external axis at equal intervals within E_x . After this step, the relative position of the robot system frame and world frame is fixed.
- 2) Sample the semi-constraint Cartesian path point at equal intervals within T . The sampling result is a set of Cartesian points with fixed position and orientation.

C. Inverse Kinematics Generation

To control the robot system with external axes, we need to map the point in the sampling space into the constraint manifolds. Since the external axes position and the relative position between the robot base frame and Cartesian point are fixed in the previous processes, for each sampling result, inverse kinematics is executed only for the robot to compute the corresponding joint values. Benefit from the decoupling of the robot and the external axes, the analytical method can be easily used for IK solution calculation instead of using the numerical method, which is usually two orders of magnitude slower than the analytical method. Generally, the structure of industrial robot conforms to pepper's principle, so up to eight IK solutions can be obtained through analytical methods. Remove IK solutions that collide with obstacles or exceed the joint limits, and then we can get a set of joint space configurations $c(c \in \mathbf{M})$, which satisfy the task constraints.

D. Optimal Path Search

Using the configurations generated in the previous processes, we can construct the C-space search graph $G = (V, E)$ along the workspace path, as illustrated in Fig. 3. The upper part shows the semi-constrained path in the workspace, where the size of the point indicates the tolerance scale of the

path point, and the lower part shows corresponding configurations in C-space. All vertices in V represent configurations that satisfy the task constraints, and all edge in E represent robot system motions that satisfy the kinematic constraints (e.g., velocity limits). Algorithm 1 describes the process of generating the Approximation Graph given a set of joint space configurations.

Algorithm 1 Generate Search Graph in C-Space

Input: ns : # semi-constraint path points,
 $poses$: Configurations that meet task constraints
Output: $G = (V, E)$: Search Graph in C-Space

```

1:  $G = \text{EmptyGraph}()$ 
2: for  $i = 1 \rightarrow ns - 1$  do
3:   for  $j = 0 \rightarrow poses[i - 1].size() - 1$  do
4:     for  $k = 0 \rightarrow poses[i].size() - 1$  do
5:       if  $\text{ValidEdge}(poses[i - 1][j], poses[i][k])$  then
6:          $edges.add(poses[i - 1][j], poses[i][k])$ 
7:       end if
8:     end for
9:   end for
10: end for
11:
12: for  $i = 0 \rightarrow ns - 1$  do
13:   for  $j = 0 \rightarrow poses[i].size() - 1$  do
14:      $G.add\_Vertex(poses[i][j])$ 
15:   end for
16: end for
17:
18: for  $i = 0 \rightarrow edges.size() - 1$  do
19:    $G.add\_Edge(edges[i].joint\_start, edges[i].joint\_end,$ 
20:      $edges[i].transition\_cost)$ 
21: end for
22: return  $G$ 

```

In order to get a continuous C-space path without singularity problem, the sum of joints rotation during task execution needs to be minimized. Therefore, the length of each edge in the search graph is set to the sum of the absolute value of the angle change of each joint between the two system configurations to represent the magnitude of the system motion. Assuming $pose_m, pose_n \in V$, the distance function can be represented as

$$d_{p_m, p_n} = \sum_{i=1}^6 k_i |J_{p_m}^i - J_{p_n}^i|^2 + l_{p_m} + l_{p_n} \quad (4)$$

$J_{p_m}^i$ is the angle value of the i -th joint at $pose_m$, k_i represents the weight corresponding to the i -th joint when calculating the distance, l_{p_m} represents the loss function that can be defined according to the actual application scenario. In industrial applications, the distance between vertices can be flexibly defined according to specific requirements. For example, for a typical industrial robot, the first two links are larger than other links. Therefore, when defining the distance in the joint space, we can set large weights to joints that connect large links. Finally, the dynamic programming algorithm is used to search for the optimal path in the joint space search graph, as described in algorithm 2.

Algorithm 2 Optimal Path Search in C-Space

Input: $G = (V, E)$: Search Graph in C-Space,
 $transition_cost$: distance function, s : virtual source point
Output: Optimal Path in C-Space

```

1: //init
2: for each vertex  $v \in G.V$  do
3:    $v.dist = INF$ 
4:    $v.prev = NULL$ 
5: end for
6:  $s.dist = 0$ 
7:
8: //traverse the vertex pathwise
9: for each vertex  $u$  in  $G$ , taken in order do
10:  for each vertex  $v \in G.Adj[u]$  do
11:    if  $v.d > u.d + transition\_cost(u, v)$  then
12:       $v.d = u.d + transition\_cost(u, v)$ 
13:       $v.prev = u$ 
14:    end if
15:  end for
16: end for
17:
18: return  $G$ 

```

The time complexity of the search algorithm is $O(V + E)$, which is better than the conventional shortest path search algorithm such as Dijkstra $O(E \log V)$ and Bellman-Ford $O(VE)$, and can get better performance when the search graph is large.

IV. EXPERIMENTS

A. Task Constraints

We use two sets of constraints to evaluate the proposed algorithm's performance under the typical Cartesian curve tracking tasks, and their text description as follows:

1) *Cylindrical intersecting curve tracking task*: When using industrial robots to transport goods or welding large objects, the length of the robot's arm limits the range of the robot's working space. Usually, the robot can be installed on the gantry to solve this problem. In this constraint, the robot system with external linear axes (gantry with 2 DOFs) needs to track a fixed cylindrical intersecting curve. And for each path point on the curve, the orientation of robot's end effector is fixed except for the tolerance around the Z_t axis.

2) *Circular curve tracking task*: When using industrial robots for spraying or cutting objects with complicated surfaces, the number of joints limits the flexibility of the system. The industry usually uses positioners or turntables to increase the flexibility of the system. In circular curve tracking task constraints, the robot system with an external rotation axis (1 DOF) needs to track the

circular curve on the tube. The orientation constraints for robot's end-effector are the same as constraint 1. The difference from constraint 1 is that the curve is not fixed but will rotate with the external rotation axis. This is useful, for example, for applications like laser cutting where the interruption of motion will lead to a reduction in product quality.

B. Implementation Details

Our method is implemented under the MoveIt! framework in ROS [15] and uses FCL [16] for collision checks. Computation of the forward kinematics for single configurations is done using the KDL library [17]. Computation of the IK for single samples is done using the analytical method generated by the IKfast [18]. We use different industrial robot models for different constraints, ABB IRB120 for constraint1, ABB IRB1200 for constraints2. All experiments were performed on a system with $2.5\text{GHz} \times 2$ Intel i5-7300HQ CPU and 6 GB RAM.

C. Results

In this section, we present the experimental results under different task constraints.

Fig. 4 shows the execution process of the ABB IRB120 robot installed on a 2 DOFs gantry under cylindrical intersecting curve tracking task. The corresponding changes in joint angles and gantry positions are shown in Fig. 5.

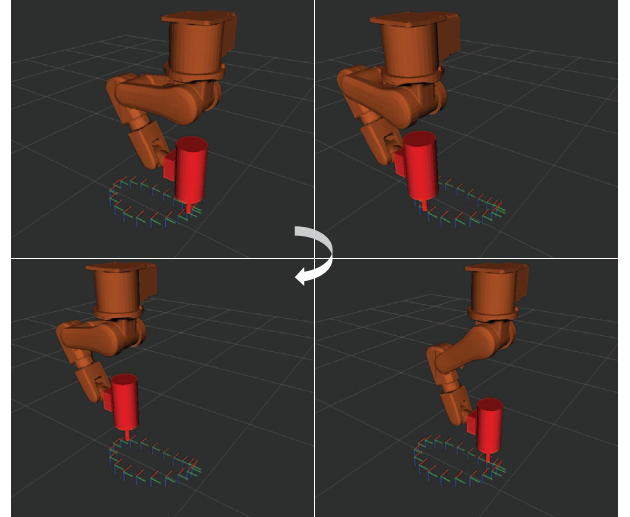


Fig. 4. Execution process of ABB IRB120 robot installed on a 2 DOFs gantry under constraint 1

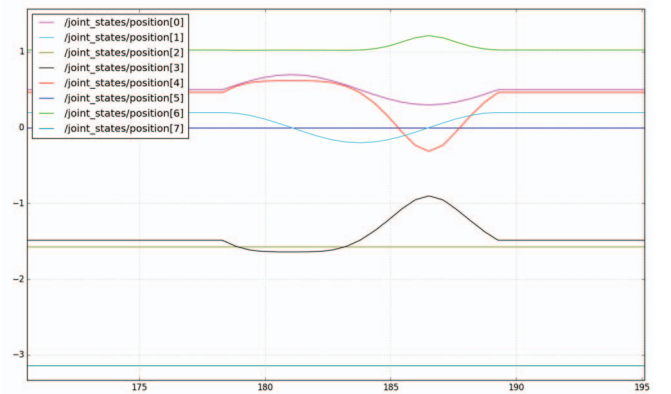


Fig. 5. Joint value and gantry position changes under constraint 1

Similarly, Fig. 6 shows the execution process of the ABB IRB1200 robot with an external rotation axis under a circular curve tracking task. And changes in joint angles and rotation axis are shown in Fig. 7.

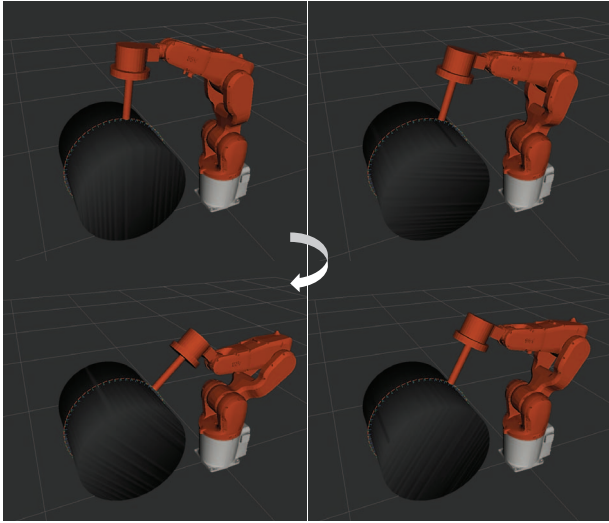


Fig. 6. Execution process of ABB IRB1200 robot with external rotation axis under constraint 2

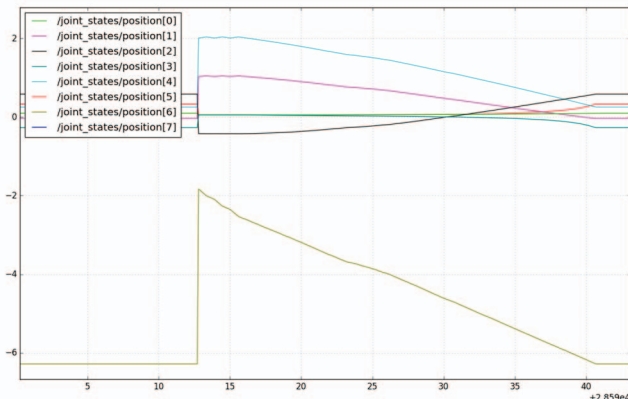


Fig. 7. Joint value and rotation angle changes under constraint 2

We can see that the proposed method can generate a continuous path for the semi-constraint path planning problem without singularity problem.

To test the algorithm's performance when the sample parameters vary, we performed experiments with different Cartesian path point numbers in workspace and different sample numbers in sample space under constraint 2. The results are shown in Table 1 and Table 2.

TABLE I. INFLUENCE OF THE NUMBER OF SAMPLE POINTS

# Cartesian Points /# Sample Points	IK Generation Time/s	Path Searching Time in C-Space/s	Sum of Joints Rotation/rad
60/5	0.7	0.08	11.6
60/30	3.3	2.31	10.2
60/180	19.2	60.81	8.9

TABLE II. INFLUENCE OF THE NUMBER OF CARTESIAN PATH POINTS

# Cartesian Points /# Sample Points	IK Generation Time/s	Path Searching Time in C-Space/s	Sum of Joints Rotation/rad
30/180	7.5	22.89	8.6
60/180	19.2	60.81	8.9
90/180	27.3	86.10	9.0

From Table 1, we can see that as the number of samples at each Cartesian path point increases, the time it takes to solve inverse kinematics increases linearly. Meanwhile, the time spent on path planning in the C-space will increase in quadratic order because the increase in the number of sampling points will cause the number of vertices and the number of edges in the C-space search graph for path planning to increase at the same time. The performance index of the sum of joints rotation represents the smoothness of the robot system's motion. The smaller the joints rotation, the smoother the robot system's motion. It can be seen that as the number of samples increases, the sum of joints rotation gradually decreases.

It can be seen from Table 2 that as the number of Cartesian path points increases, the IK generation time and the C-space path search time both increase linearly, which is caused by the linear increase in the amount of calculation required. And for paths with different numbers of Cartesian path points, since they all represent the same Cartesian curve tracking task constraints, when the number of samples is large enough, the sum of joint rotation tends to be the same, indicating that the optimal path that satisfy the task constraints under the corresponding sample parameters has been found.

V. THE CONCLUSION & FUTURE WORKS

In this paper, we proposed a semi-constrained path planning algorithm for robots with external axes. Our approach can generate configurations in the constrained C-space directly. Simulation under practical task constraints was down to illustrate the effectiveness and performance of the proposed algorithm. The results show that the algorithm can generate a continuous and smooth path without singularity problem. Moreover, it can be applied to typical industrial applications with Cartesian curve tracking task constraints such as welding, laser/plasma cutting, and spraying. In future work, we plan to incorporate more task constraints, such as time constraints, torque constraints, so that the applicability of the algorithm can be further enhanced.

REFERENCES

- [1] F. Z. Baghli, L. E. Bakkali, Y. Lakhal, "Optimization of Arm Manipulator Trajectory Planning in the Presence of Obstacles by Ant Colony Algorithm," *Procedia Engineering*, 2017, 181:560-567
- [2] H. Huang and C. Tsai, "Global path planning for autonomous robot navigation using hybrid metaheuristic GA-PSO algorithm," *SICE Annual Conference 2011, Tokyo, 2011*, pp. 1338-1343.
- [3] G. Li, A. Yamashita, H. Asama and Y. Tamura, "An efficient improved artificial potential field based regression search method for robot path planning," *2012 IEEE International Conference on Mechatronics and Automation, Chengdu, 2012*, pp. 1227-1232, doi: 10.1109/ICMA.2012.6283526.

- [4] L. Jaillet, J. M. Porta. 2013. Path planning under kinematic constraints by rapidly exploring manifolds. *IEEE Trans. Robot.* 29:105–17
- [5] D. Webb, J. van den Berg. 2013. Kinodynamic RRT*: asymptotically optimal motion planning for robots with linear dynamics. In 2013 IEEE International Conference on Robotics and Automation, pp. 5054–61. New York: IEEE
- [6] B. Kim, T. T. Um, C. Suh and F. C. Park, “Tangent bundle RRT: A randomized algorithm for constrained motion planning,” *Robotica*, 2016, 34(1): 202–225.
- [7] J. X. Ge, F. C. Sun, C. F. Liu. “PRT-GD: an efficient rapidly exploring random tree approach with goal directionality for redundant manipulator path planning,” In: Proceedings of the 2016 IEEE International Conference on Robotics and Biomimetics. Qingdao, China: IEEE, 2016. 1983–1988
- [8] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al. 2005. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA: MIT Press
- [9] S. M. LaValle. 2006. *Planning Algorithms*. Cambridge, UK: Cambridge Univ. Press
- [10] Z. Kingston, M. Moll and L. E. Kavraki, “Decoupling constraints from sampling-based planners,” In: *Int. Symp. on Robotics Research*. Springer, 2017.
- [11] J. Mirabel and F. Lamiraux, “Handling implicit and explicit constraints in manipulation planning,” In: *Robotics: Science and Syst.* DOI:10.15607/RSS.2018. XIV.018.
- [12] M. Stilman, "Global Manipulation Planning in Robot Joint Space With Task Constraints," *Robotics, IEEE Transactions on*, vol. 26, pp. 576–584, 2010.
- [13] T. Kunz and M. Stilman, "Manipulation planning with soft task constraints," 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, Vilamoura, 2012, pp. 1937–1942.
- [14] G. Oriolo, M. Ottavi, and M. Vendittelli, "Probabilistic motion planning for redundant robots along given end-effector paths," in *Intelligent Robots and Systems. IEEE/RSJ International Conference on*, 2002, pp. 1657–1662.
- [15] S. Chitta, I. Sucan, and S. Cousins, “MoveIt! [ROS topics],” *IEEE Robotics Automation Magazine*, vol. 19, no. 1, pp. 18 –19, 2012.
- [16] J. Pan, S. Chitta, and D. Manocha, “FCL: A general purpose library for collision and proximity queries,” in *Proc. of the IEEE Int. Conf. on Robotics & Automation (ICRA)*, 2012.
- [17] R. Smits, “KDL: Kinematics and Dynamics Library,” <http://www.orocos.org/kdl>.
- [18] R. Diankov, “IKFast: The Robot Kinematics Compiler” http://www.openrave.org/docs/latest_stable/openravepy/ikfast/